Task 1: Decode a Nested User

```
import Foundation
struct User: Decodable {
    let id: Int
    let name: String
    let address: Address
    let company: Company
}
struct Address: Decodable {
    let city: String
    let geo: Geo
}
struct Geo: Decodable {
    let lat: String
    let lng: String
}
struct Company: Decodable {
    let name: String
}
let json = """
{
    "id": 1,
    "name": "Leanne Graham",
    "address": {
        "city": "Gwenborough",
```

```
        "geo": {
            "lat": "-37.3159",
            "lng": "81.1496"
        }
    },
    "company": {
        "name": "Romaguera-Crona"
    }
}
"""
do {
    let user = try JSONDecoder().decode(User.self, from: Data(json.utf8))
    print("Name: \(user.name)")
    print("City: \(user.address.city)")
    print("Company: \(user.company.name)")
} catch {
    print("Decoding error: \(error)")
}
```

Output

```
Name: Leanne Graham
City: Gwenborough
Company: Romaguera-Crona
```

Task 2: Decode Books with Snake Case, Dates and Optionals

```
import Foundation
struct Book: Decodable {
   let bookID: Int
   let bookTitle: String
   let publishedOn: Date
   let rating: Double?
}

let json = """
[
   {"book_id": 1, "book_title": "Swift Basics", "published_on": "2024-03-15", "rating": 4.5},
   {"book_id": 2, "book_title": "UIKit Guide", "published_on": "2023-11-01"}
]
"""
let decoder = JSONDecoder()
decoder.keyDecodingStrategy = .convertFromSnakeCase

let formatter = DateFormatter()
formatter.dateFormat = "yyyy-MM-dd"
formatter.locale = Locale(identifier: "en_US_POSIX")
formatter.timeZone = TimeZone(secondsFromGMT: 0)
decoder.dateDecodingStrategy = .formatted(formatter)

do {
```

```
    let books = try decoder.decode([Book].self, from: Data(json.utf8))
    for book in books {
        if let rating = book.rating {
            print("\(book.bookTitle): \(rating)")
        } else {
            print("\(book.bookTitle): no rating")
        }
    }
} catch {
    print("Decoding error: \(error)")
}
```



Task 3: Handle JSON Decoding Errors and Accept Both ID Types

```
import Foundation
struct Item: Decodable {
    let id: Int
```

```
    let name: String

    enum CodingKeys: String, CodingKey {
        case id
        case name
    }
    init(from decoder: Decoder) throws {
        let container = try decoder.container(keyedBy: CodingKeys.self)

        if let intID = try? container.decode(Int.self, forKey: .id) {
            id = intID
        } else if let stringID = try? container.decode(String.self, forKey: .id),
                  let convertedID = Int(stringID) {
            id = convertedID
        } else {
            throw DecodingError.dataCorruptedError(
                forKey: .id,
                in: container,
                debugDescription: "ID must be an integer or a numeric string"
            )
        }

        name = try container.decode(String.self, forKey: .name)
    }
}
```

```
let inputs = [
    #"{"id": 1}"#,
    #"{"id": "5", "name": "Pen"}"#,
    #"{"id": 5, "name": "Book"}"#
]
for (index, input) in inputs.enumerated() {
    do {
        let item = try JSONDecoder().decode(Item.self, from: Data(input.utf8))
        print("Item: \(item.id), \(item.name)")
    } catch {
        print("Input \(index + 1): Invalid JSON data - \(error.localizedDescription)")
    }
}
```

